

Data Analysis with Python 2024–2025

Parte 2: Python Caderno de Exercícios

Tradução e Adaptação: University of Helsinki — MOOC.fi

Nota Importante

Este material é uma tradução e adaptação baseada no curso *Data Analysis with Python 2024-2025*, oferecido pela **Universidade de Helsinque (MOOC.fi)**.

Conteúdo

1 Exercícios - Capítulo 2

Exercício 2.1 – Inteiros entre colchetes

Sumário

- **Objetivo:** Escrever uma função `integers_in_brackets(s)` que retorne uma lista de todos os números inteiros que estejam isolados dentro de colchetes na string `s`.
- **Restrições:** Pode haver espaços em branco entre os colchetes e os números, mas não pode haver nenhum caractere não-numérico (além do possível sinal de `+` ou `-`). Utilize expressões regulares (RegEx).

Exercício 2.2 – Listagem de arquivos

Sumário

- **Objetivo:** Escrever uma função `file_listing(caminho)` que faça o parsing do arquivo `listing.txt`.
- **Requisito:** O arquivo texto imita a saída do comando Linux `ls -l`. Cada linha possui permissões, links, usuário, grupo, **tamanho**, **mês**, **dia**, **hora:minuto**, **nome do arquivo**.
- **Retorno esperado:** Uma lista contendo as tuplas formatadas da seguinte forma: `(tamanho (int), mês (str), dia (int), hora (int), minuto (int), nome (str))`. Use `re.match` ou `re.findall`.

Exercício 2.3 – Vermelho, verde, azul (RGB)

Sumário

- **Objetivo:** Criar uma função `red_green_blue(caminho)` para limpar e processar o arquivo `rgb.txt`.
- **Procedimento:** Remova a primeira linha do arquivo (que é inútil para a análise). O restante das linhas deve ser filtrado.
- **Saída:** Uma lista de strings, onde cada linha original é formatada para conter apenas os quatro campos originais (**R**, **G**, **B**, **Nome da Cor**), separados por exatamente uma tabulação (`\t`). Utilize RegEx para limpar excessos de espaços.

Exercício 2.4 – Frequência de palavras

Sumário

- **Objetivo:** Escrever `word_frequencies(nome_arquivo)` para contar as palavras do texto (ex: `alice.txt`).
- **Limpeza:** Quebre cada linha em palavras usando `split()`. Em seguida, use o comando `strip(PONTUACAO)` para remover sinais gráficos soltos ao redor de cada palavra.
- **Saída:** Retorne um dicionário (`dict`) onde as chaves são as palavras limpas e os valores são as quantidades absolutas de ocorrências.

Exercício 2.5 – Resumo estatístico

Sumário

- **Objetivo:** Desenvolver `summary(nome_arquivo)` que calcule três medidas descritivas básicas de um arquivo contendo um número flutuante por linha.
- **Cálculos:** Retornar uma tupla contendo a **soma**, a **média** e o **desvio padrão amostral** ($N - 1$).
- **Exceções:** Se o programa encontrar uma linha que não puder ser convertida para float, deve usar um bloco `try... except ValueError` para ignorar a linha de forma segura e continuar lendo o arquivo.
- **Integração:** Crie um `main()` para invocar sua função iterando sob o array `sys.argv[1:]` recebido por terminal.

Exercício 2.6 – Contagem de arquivo

Sumário

- **Objetivo:** Criar a função `file_count(nome_arquivo)` para realizar análises de quantidade em um arquivo.
- **Saída:** Tupla com o (número de linhas, número de palavras, e número total de caracteres).
- **Terminal:** Crie o loop em `main()` iterando por `sys.argv` para imprimir na tela, com separações via tabulação (`\t`).

Exercício 2.7 – Extensões de arquivo

Sumário

- **Objetivo:** Escrever a função `file_extensions(nome_arquivo)` para separar arquivos lidos de um TXT.
- **Estruturação:** Percorra as linhas e verifique se o arquivo possui extensão lendo a posição após o último ponto.
- **Saída:** A função retornará 2 elementos. (1) Uma lista com os nomes de arquivos que não possuem extensões. (2) Um dicionário que tem a **extensão** como chave, contendo listas com os **nomes dos respectivos arquivos** como valores.

Exercício 2.8 – Prepend

Sumário

- **Objetivo:** Aplicar orientação a objetos (OO) ao criar a classe `Prepend`.
- **Inicialização:** A classe deve ter um inicializador `__init__` que armazene no escopo `self` a string inicializadora.
- **Método:** A classe deve ter o método de instância `write(s)`, que fará o `print` unindo o atributo interno `self.start` mais o parâmetro `s`.

Exercício 2.9 – Números racionais

Sumário

- **Objetivo:** Construir e sobrecarregar de métodos a classe matemática `Rational` para suportar o formato fracionário real.
- **Matemática:** O racional é inicializado recebendo o numerador e denominador. Devem ser sempre salvos em sua fração mais simplificada possível (use o método nativo de Máximo Divisor Comum `math.gcd`).
- **Sobrecarga de Métodos Dunder:** A classe deve sobrescrever os operadores básicos: `__add__` (+), `__sub__` (-), `__mul__` (*), `__truediv__` (/).
- **Sobrecarga Lógica:** Também deve sobrescrever os comparadores (`__eq__`, `__gt__`, `__lt__`).

Exercício 2.10 – Extrair números

Sumário

- **Objetivo:** Implementar `extract_numbers(s)` que retira os dados numéricos (salvos em texto misturados à palavras) em seus respectivos tipos reais.
- **Processo Dúplice:** Tente usar o *casting* `int()` sob as partições do texto (usando `try`). Se der erro (`except`), crie outra tentativa para convertê-la para o formato `float()`. Caso caia no segundo `except`, é porque é uma letra/palavra irrelevante, então ignore-a.
- **Retorno:** Uma lista preenchida apenas com Inteiros e Floats matematicamente corretos extraídos da string de base.